

Top 10 Key Takeaways

Distilled from the diarized rough-cut transcript, Chunks 1–9 (~14:04–15:33). Instructors: Madeleine (MW), Marlon (MK), with faculty demos from Jungyoon, and the room including Jeff Behrends, Natasha Sumner, and Jason.

- 1

The class-summarizer demo is the move — every recording you have is now a transcript a chat can interrogate. Marlon copy-pastes the path to the day's transcript and asks for *"the top three most important takeaways."* Every faculty member already has hours of recorded classes piling up — *"they're probably just taking you all the time, whether you want it or not."* Hand the transcript to a student who missed class. Hand it to yourself when prepping next week's session.
- 2

The triptych — inputs/ → operations/ → outputs/ — is the recipe pattern, and it's how you start every new project. *"We're just gonna keep hitting this."* Faculty building from scratch: open Claude and say *"in the examples folder please make me a new example, please mirror the folder structure of the other documents, make sure there's inputs, operations and outputs."* Recipe cards from Day 1, now load-bearing structure.
- 3

Keep AI-generated outputs separated from human-authored content — the outputs/ folder is the firebreak. Marlon's index-card analogy: *"there were a lot of plagiarism controversies when I was young for people working on index cards — they forgot whether that index card was a note I had copied from a text or if I really wrote it."* Same problem with fifty thousand floating markdown docs. *"As soon as you're dealing with AI generation based on AI generation based on AI generation, there's gonna be a falloff."*
- 4

Don't substitute Claude's fluency for your own expertise. MW, to a faculty member starting to lean on Claude for assessment design: *"It gets very tempting because Claude seems so smart and fluent and capable at things we can't do as quickly to start substituting its intelligence for our own. But don't forget that you're the expert as a teacher and in your discipline."* The whole workshop's posture toward AI in three sentences.
- 5

The paper-to-teaching-materials pattern, with Jeff Behrends in the room. Start with grounded material you trust — a paper you wrote, a paper you teach — and generate the *class* out of it: a teaching case, a discussion plan, an objection audit that anticipates student arguments, a ten-to-fifteen-minute quiz. *"You wouldn't stop here, but this is a great starting point."* The audit step is the discipline: the *instructor* decides whether the output is good enough for their classroom.

Takeaways 6–10

"The spell book," CLAUDE.md vs. skills, and where skills actually live on disk.

- 6

The grade-policy demo (Jungyoon's recentering-academics example) is "the spell book." A folder of source material — Bok advice, Harvard's grade-inflation reports, the external research on grading — plus a prompt that asks Claude to recommend changes to *your* syllabus or *your* concentration description against that evidence. MW: *"This is like the spell book — it's not gonna magically make everything perfect, but it's a great place to start. You can keep adding to this with your own educational research or previous courses you really liked."*
- 7

CLAUDE.md = truths. Skills = behaviors. A CLAUDE.md is *always* loaded into context (the limerick CLAUDE.md from yesterday — *every* output was a limerick because that one sentence sat at the top of every chat). MW's rule: *"make sure these are absolute truths,"* keep it under ~200 lines. Skills are **conditional** — loaded only when Claude judges the task matches the skill's description.
- 8

CLAUDE.md walks up the file tree, and the global one has primacy. MW: *"Claude walks up your file structure to look for all the CLAUDE MDs."* Project CLAUDE.md loads first, then the development-folder one (if any), then *~/.claude/CLAUDE.md* (global). The global gets focused on most often — put genuinely cross-cutting preferences there, not per-project rules. *"I would never put a CLAUDE.md that says only code with R at the root, because I'm constantly working in JS, Python, etc., and it will really follow that."*
- 9

Skills are SOPs — recurrent processes you want done exactly the same way every time. MK: *"the recurrent processes you have to perform — formatting, transcription, things you've gotta do again and again — you wanna do them exactly the same way every time. In industry they talk about SOPs, standard operating procedures."* His sonnet-reader example: every Shakespeare sonnet analyzed into the same fields (rhyme scheme, type, year, author) so they land in an Excel-able spreadsheet. *"I don't just want Claude to vibe that anew for every single sonnet."*
- 10

Desktop-app skills save to the cloud; Claude Code skills save locally to .claude/. Anthropic is candidly still figuring this out. MK: *"Anthropic is confused about what to do here — they're trying to make this user-friendly, but it actually is legit confusing what they want."* Desktop-app skill → lives in your account in the cloud, follows you to any machine. CLI / VS Code skill → lives in *.claude/skills/* in either the project root (project-scoped) or *~/.claude/* (user-scoped, global). A skills *marketplace* for orgs is on the roadmap but not yet live.

SECONDARY POINTS WORTH KEEPING

- **Start a new example by copying an existing one.** *"you could copy the entire folder itself in a new project, and say hey, this is from philosophy, but I'm teaching in X department, can you duplicate it with this new PDF."* Faster than writing structure from scratch.
- **The Dylan and Joyce examples are "close reading at scale."** MK: *"You can't use simple word search to find every author in the lyrics — authors have thousands of names. That's what LLMs are cool for."* Joyce's puns on Finn MacCool: 277 references. *"Faster than a summer research intern would be."*
- **Force-trigger a skill with /skillname.** MW: *"If I don't trust Claude to just pick it up by vibes, I can type slash and then the word skill-creator, and force Claude to use that skill."* The manual override on Claude's relevance judgment.
- **Skills can carry scripts, not just text.** Shell, Python, JavaScript. MK: *"we may take a video and extract the audio from it and then send that audio to a transcription service and then summarize that."* (Sub-agents got a brief preview — more tomorrow.)